

Week 2

1.

```
1 import java.util.Arrays;
2 class Solution {
3     public static int kthSmallest(int[] arr, int k) {
4         Arrays.sort(arr);
5         return arr[k - 1];
6     }
7     public static void main(String[] args) {
8         int[] arr1 = {10, 5, 4, 3, 48, 6, 2, 33, 53, 10};
9         int k1 = 4;
10        System.out.println(kthSmallest(arr1, k1));
11        int[] arr2 = {7, 10, 4, 3, 20, 15};
12        int k2 = 3;
13        System.out.println(kthSmallest(arr2, k2));
14    }
15 }
```

1.

```
1 class Solution {
2     public int getMinDiff(int[] arr, int k) {
3         int n = arr.length;
4         Arrays.sort(arr);
5         int ans = arr[n - 1] - arr[0];
6         int smallest = arr[0] + k;
7         int largest = arr[n - 1] - k;
8         for (int i = 1; i < n; i++) {
9             if (arr[i] - k < 0)
10                continue;
11            int minHeight = Math.min(smallest, arr[i] - k);
12            int maxHeight = Math.max(largest, arr[i - 1] + k);
13            ans = Math.min(ans, maxHeight - minHeight);
14        }
15        return ans;
16    }
17 }
```

1.

```
1 class Solution {
2     public int minJumps(int[] arr) {
3         int n = arr.length;
4         if (n == 1)
5             return 0;
6         if (arr[0] == 0)
7             return -1;
8         int maxReach = arr[0];
9         int steps = arr[0];
10        int jumps = 1;
11        for (int i = 1; i < n; i++) {
12            if (i == n - 1)
13                return jumps;
14            maxReach = Math.max(maxReach, i + arr[i]);
15            steps--;
16            if (steps == 0) {
17                jumps++;
18                if (i >= maxReach)
19                    return -1;
20                steps = maxReach - i;
21            }
22        }
23        return -1;
24    }
25 }
```

1.

```
1 class Solution {
2     public int findDuplicate(int[] nums) {
3         int slow = nums[0];
4         int fast = nums[0];
5         do {
6             slow = nums[slow];
7             fast = nums[nums[fast]];
8         } while (slow != fast);
9         slow = nums[0];
10        while (slow != fast) {
11            slow = nums[slow];
12            fast = nums[fast];
13        }
14        return slow;
15    }
}
```

1.

```
1 class Solution {
2     private int nextGap(int gap) {
3         if (gap <= 1)
4             return 0;
5         return (gap / 2) + (gap % 2);
6     }
7     public void mergeArrays(int[] a, int[] b) {
8         int n = a.length;
9         int m = b.length;
10        int gap = nextGap(n + m);
11        while (gap > 0) {
12            int i = 0;
13            int j = gap;
14            while (j < n + m) {
15                if (i < n && j < n) {
16                    if (a[i] > a[j]) {
17                        int temp = a[i];
18                        a[i] = a[j];
19                        a[j] = temp;
20                    }
21                }
22                else if (i < n && j >= n) {
23                    if (a[i] > b[j - n]) {
24                        int temp = a[i];
25                        a[i] = b[j - n];
26                        b[j - n] = temp;
27                    }
28                }
29                else {
30                    if (b[i - n] > b[j - n]) {
31                        int temp = b[i - n];
32                        b[i - n] = b[j - n];
33                        b[j - n] = temp;
34                    }
35                }
36                i++;
37                j++;
38            }
39            gap = nextGap(gap);
40        }
41    }
42 }
```

1.

```
1 import java.util.*;
2 class Solution {
3     public int[][] merge(int[][] intervals) {
4         if (intervals.length <= 1)
5             return intervals;
6         Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
7         List<int[]> result = new ArrayList<>();
8         int[] current = intervals[0];
9         result.add(current);
10        for (int i = 1; i < intervals.length; i++) {
11            int[] next = intervals[i];
12            if (next[0] <= current[1]) {
13                current[1] = Math.max(current[1], next[1]);
14            }
15            else {
16                current = next;
17                result.add(current);
18            }
19        }
20        return result.toArray(new int[result.size()][]);
21    }
22 }
```

2.

```
1 import java.util.*;
2 class Solution {
3     public List<Integer> commonElements(int[] arr1, int[] arr2, int[] arr3) {
4         int i = 0, j = 0, k = 0;
5         List<Integer> result = new ArrayList<>();
6         while (i < arr1.length && j < arr2.length && k < arr3.length) {
7             if (arr1[i] == arr2[j] && arr2[j] == arr3[k]) {
8                 result.add(arr1[i]);
9                 int val = arr1[i];
10                while (i < arr1.length && arr1[i] == val) i++;
11                while (j < arr2.length && arr2[j] == val) j++;
12                while (k < arr3.length && arr3[k] == val) k++;
13            }
14            else if (arr1[i] < arr2[j]) {
15                i++;
16            }
17            else if (arr2[j] < arr3[k]) {
18                j++;
19            }
20            else {
21                k++;
22            }
23        }
24        if (result.isEmpty()) {
25            result.add(-1);
26        }
27        return result;
28    }
29 }
```

2.

```
1 import java.util.*;
2 class Solution {
3     public static ArrayList<Integer> factorial(int n) {
4         ArrayList<Integer> result = new ArrayList<>();
5         result.add(1);
6         for (int x = 2; x <= n; x++) {
7             multiply(result, x);
8         }
9         Collections.reverse(result);
10        return result;
11    }
12    private static void multiply(ArrayList<Integer> result, int x) {
13        int carry = 0;
14        for (int i = 0; i < result.size(); i++) {
15            int prod = result.get(i) * x + carry;
16            result.set(i, prod % 10);
17            carry = prod / 10;
18        }
19        while (carry > 0) {
20            result.add(carry % 10);
21            carry /= 10;
22        }
23    }
24 }
```

2.

```
1 import java.util.*;
2 class Solution {
3     public boolean isSubset(int[] a, int[] b) {
4         if (b.length > a.length) {
5             return false;
6         }
7         HashMap<Integer, Integer> freq = new HashMap<>();
8         for (int x : a) {
9             freq.put(x, freq.getOrDefault(x, 0) + 1);
10        }
11        for (int x : b) {
12            if (!freq.containsKey(x) || freq.get(x) == 0) {
13                return false;
14            }
15            freq.put(x, freq.get(x) - 1);
16        }
17        return true;
18    }
19 }
```

2.

```
1 import java.util.*;
2 class Solution {
3     public boolean hasTripletSum(int[] nums, int target) {
4         int n = nums.length;
5         Arrays.sort(nums);
6         for (int i = 0; i < n - 2; i++) {
7             int left = i + 1;
8             int right = n - 1;
9             while (left < right) {
10                int sum = nums[i] + nums[left] + nums[right];
11
12                if (sum == target) {
13                    return true;
14                } else if (sum < target) {
15                    left++;
16                } else {
17                    right--;
18                }
19            }
20        }
21        return false;
22    }
23 }
```


2.

```
1 class Solution {
2     public long maxWater(int[] arr) {
3         int n = arr.length;
4         int left = 0, right = n - 1;
5         int leftMax = 0, rightMax = 0;
6         long water = 0;
7         while (left <= right) {
8             if (arr[left] <= arr[right]) {
9                 if (arr[left] >= leftMax) {
10                     leftMax = arr[left];
11                 } else {
12                     water += leftMax - arr[left];
13                 }
14                 left++;
15             } else {
16                 if (arr[right] >= rightMax) {
17                     rightMax = arr[right];
18                 } else {
19                     water += rightMax - arr[right];
20                 }
21                 right--;
22             }
23         }
24         return water;
25     }
26 }
```